LAB 06

6 Bài 1:

$x_t = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$, $h_{t-1} = \begin{bmatrix} 0,5 \\ 0,2 \end{bmatrix}$, $W_{xh} = \begin{bmatrix} 0,5 & 0,2 \\ 0,1 & 0,4 \end{bmatrix}$, $W_{hh} = \begin{bmatrix} 0,3 & 0,1 \\ 0,2 & 0,5 \end{bmatrix}$

$b_h = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$

$$a_t = W_{xh} x_t + W_{hh} h_{t-1}$$
$$= \begin{bmatrix} 0,5 & 0,2 \\ 0,1 & 0,4 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \end{bmatrix} + \begin{bmatrix} 0,3 & 0,1 \\ 0,2 & 0,5 \end{bmatrix} \begin{bmatrix} 0,5 \\ 0,2 \end{bmatrix}$$
$$= \begin{bmatrix} 0,5 \\ 0,1 \end{bmatrix} + \begin{bmatrix} 0,17 \\ 0,2 \end{bmatrix} = \begin{bmatrix} 0,67 \\ 0,3 \end{bmatrix}$$
$$h_t = \tanh(a_t) = \begin{bmatrix} \tanh(0,67) \\ \tanh(0,3) \end{bmatrix} \approx \begin{bmatrix} 0,585 \\ 0,2913 \end{bmatrix}$$

7 Bài 2: "I did not like this movie".

Khi đọc từng token, hidden state được cập nhật và mang thông tin từ các token trước đó.

"I": ghi nhận chủ thể

"did": bổ sung thông tin về cấu trúc câu.

"not": ghi nhận thông tin phủ định.

"like": kết hợp thông tin trước đó, có thể hình thành ý nghĩa phủ định của "not like"

"this": tiếp tục cập nhật ngữ cảnh

"movie": hoàn thiện thông tin về đối tượng được đánh giá.

- Hidden state tại thời điểm "like" có thể kết hợp thông tin phủ định từ "not", giúp mô hình nhận biết ý nghĩa khác với "like" thông thường.

- Average Word2Vec lấy trung bình các vector từ nên không mô hình hoá trực tiếp thứ tự từ. RNN cập nhật hidden state tuần tự, nhờ đó có thể biểu diễn ngữ cảnh và ảnh hưởng của các từ trước

8 Bài 3: S_1 "dog bites man" S_2: "man bites dog"

- Average Word2Vec không phân biệt được hai câu dựa trên thứ tự do 2 câu có cùng tập từ và mỗi từ được tính trung bình như nhau nên vectơ trung bình giống nhau, giả sử dùng cùng embedding cho mỗi từ.

- RNN có thể phân biệt bằng cách đọc các từ theo thứ tự và cập nhật hidden state sau mỗi token.

Với S_1, mô hình xử lý lần lượt "dog" → "bites" → "man".

Được quét bằng CamScanner

Với S2, mô hình xử lý lần lượt "man" → "bites" → "dog". Do hidden state phụ thuộc vào token hiện tại và hidden state trước đó, trạng thái cuối có thể khác nhau. Vì vậy, RNN có khả năng phân biệt hai câu dựa trên thứ tự từ

9 Vanishing gradient

Sequence càng dài có thể khiến RNN khó học dependency ở xa vì khi huấn luyện RNN bằng backpropagation through time, gradient được truyền ngược qua nhiều timestep. Gradient có thể chứa tích của nhiều đạo hàm liên tiếp.

- Vanishing gradient: nếu các hệ số trong tích có độ lớn nhỏ hơn 1, gradient có thể giảm dần về gần 0. Các timestep ở xa nhận được gradient rất nhỏ → mô hình khó học mối quan hệ giữa những từ cách xa nhau.
- Exploding gradient: nếu các hệ số có độ lớn quá lớn, gradient có thể tăng rất nhanh, gây mất ổn định trong quá trình huấn luyện

Prediction

10 (P1): Với RNN + Embedding, mô hình xử lý word order trực tiếp hơn. Do RNN đọc từng token theo thứ tự và cập nhật hidden state dựa trên token hiện tại cùng hidden state trước đó.

Cons: Cao

(P2) Vs LSTM, mô hình xử lý dependency dài tốt hơn RNN

Do LSTM sử dụng cell state và các gates để kiểm soát việc giữ, thêm và xuất thông tin, giúp cải thiện khả năng học phụ thuộc dài hạn so với RNN thông thường.

Cons: Cao

(P3): Sequence length tăng từ 50 lên 200, training của RNN có xu hướng chậm hơn và tốn nhiều tài nguyên tính toán hơn. Do RNN phải thực hiện nhiều bước recurrent hơn cho một sequence; gradient cũng phải truyền qua nhiều timestep hơn, làm tăng nguy cơ vanishing hoặc exploding gradient.

Cons: Cao

(P4) Nếu truncate toàn bộ review về 30 tokens, thông tin classification có thể bị ảnh hưởng. Do các từ ~~ngữ~~ hoặc ngữ cảnh quan trọng nằm ngoài 30 tokens đầu sẽ bị loại bỏ, chẳng hạn từ phủ định hoặc từ thể hiện sentiment xuất hiện ở cuối review → khiến mô hình dự đoán sai

Cons: Cao

Được quét bằng CamScanner

17 $f_t = 0,9$ $i_t = 0,2$ $\tilde{C}_t = 0,5$ $C_{t-1} = 0,8$

$C_t = f_t \cdot C_{t-1} + i_t \tilde{C}_t = 0,9 \cdot 0,8 + 0,2 \cdot 0,5 = 0,82$

Nếu forget gate gần 1, mô hình có xu hướng giữ lại phần lớn thông tin cũ trong cell state

gần 0 loại bỏ

Được quét bằng CamScanner